BINARY SEARCH TREES — PART 2

"Deleting is harder than inserting — and a lazy BST becomes a linked list in disguise."

▪ TODAY'S CLASS ▪

Last class we added and searched in a BST. Both were easy. Today we deal with the two messy halves of the story: **deleting a node**, and **what happens when the tree goes wrong**.

ID	What we cover
Hook	Three cases of deletion
Case 1	Leaf — just remove
Case 2	One child — promote it
Case 3	Two children — the inorder successor trick
The code	Recursive `delete` in Python
Dark side	Degenerate BST = linked list
Self-balancing	AVL, Red-Black (concept only)

"Three cases for delete. One trick for the hard one. Then we look at what breaks when the tree is bad."

REMOVING A NODE

Inserting a value into a BST is easy — you walk to an empty spot and plant a node. Deleting is messier. The node you remove may be in the middle of the tree, and the BST property must still hold afterwards.

Three cases, by how many children the node has:

Case 1 — no children (leaf) → *easy.*

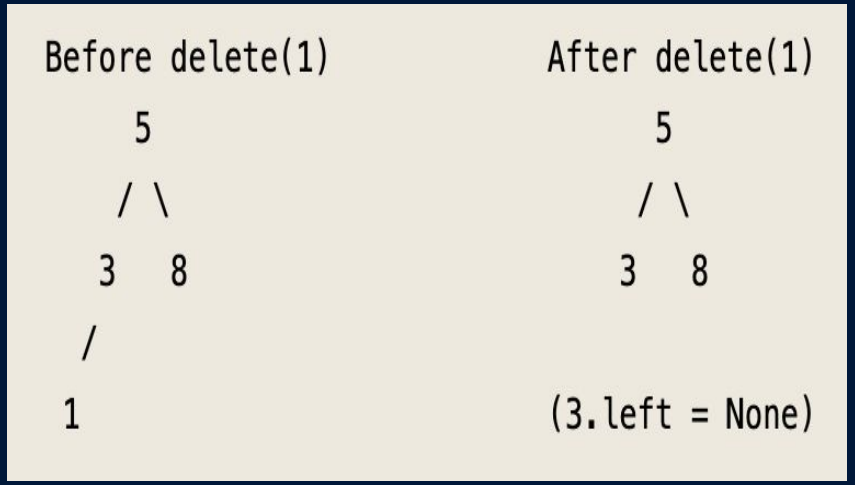
Case 2 — one child → *easy, promote it.*

Case 3 — two children → *awkward. Who takes their place?*

"If they have two friends below, the question is: which one inherits the seat?"

CASE 1 — LEAF

The easiest case. The node has no children. We disconnect it from its parent and we are done. The BST property cannot break — nothing depended on this node.




```
# In code:
if node.left is None and node.right is None:
    return None
```

The parent's recursive re-attach erases the link.

"A leaf carries no consequences. Cut it off and the tree is unchanged elsewhere."

CASE 2 — ONE CHILD

The node has exactly one child — left or right, not both. The child takes the parent's place. The grandparent now points to the grandchild.

Order is preserved because all values in that subtree were already on the correct side.


```
# In code:
if node.left is None:
    return node.right
if node.right is None:
    return node.left
```

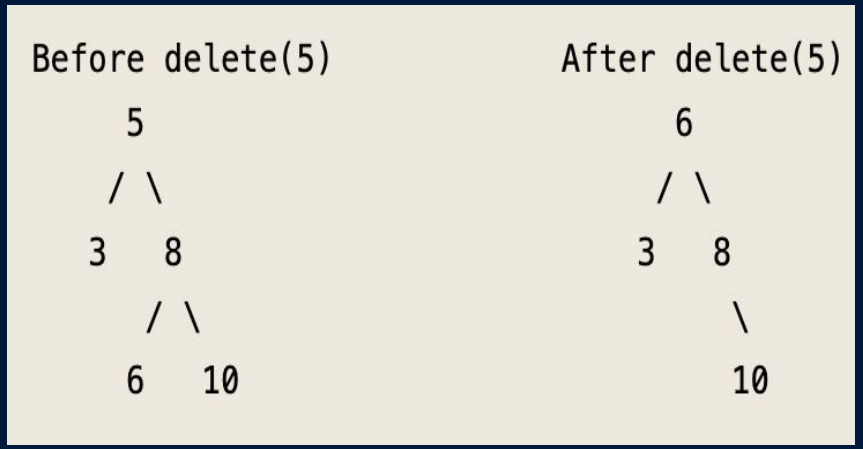
The single child jumps up into the deleted node's slot.

"One child? Promote it. The BST property already says it belongs on that side."

CASE 3 — TWO CHILDREN – INORDER SUCCESSOR

The node has both a left and a right subtree. We cannot just promote one — what about the other? The trick: replace the node's **value** with its **inorder successor** — the smallest value in its right subtree.

The node stays; only the value changes.



Step 1: Find inorder successor of 5, Go right to 8, then left to 6 → smallest in right subtree = 6.

Step 2: Copy 6 into the slot of 5.

Step 3: Delete the original 6 (now a leaf — Case 1).

"Find the next-larger value. Copy it up. Delete it from below. Order preserved, no awkward choice."

▪ WHY IT IS SAFE ▪

The inorder successor is the smallest value that is still greater than the deleted node. Copying it into the slot keeps every BST invariant intact, on both sides.

[01] Left-Subtree

All left-subtree values stay smaller.

The successor is greater than the deleted node, so it is greater than every value in the left subtree.

[02] Right-Subtree

All right-subtree values stay greater.

The successor was the smallest value of the right subtree. After we remove it, every other right-subtree value is still larger.

[03] Easy Delete

The replacement delete is always easy.

The successor is the leftmost node of the right subtree — so it has no left child. Removing it is always case 1 or case 2.

"Pick the next-larger value and the order rule survives on both sides — every time."

DELETE — THE FULL FUNCTION

One function, three cases, all recursive. The outside shape is the same as insert: walk down comparing, re-attach the returned subtree on the way back up. The inside handles the three cases when we find the node.

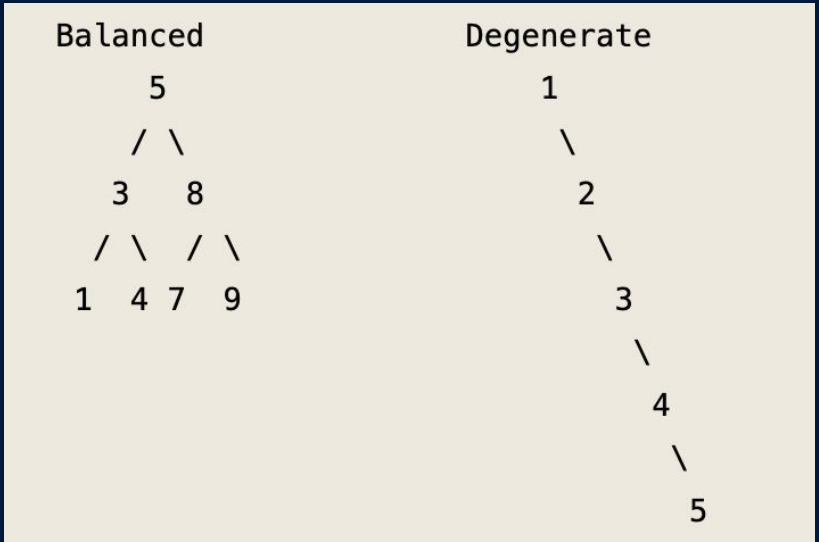
```
def delete(node, value):
    if node is None: # nothing to delete
        return None

    if value < node.value:
        node.left = delete(node.left, value)
    elif value > node.value:
        node.right = delete(node.right, value)
    else: # found it — three cases
        if node.left is None and node.right is None:
            return None # case 1: leaf
        if node.left is None: return node.right # case 2: only right child
        if node.right is None: return node.left # case 2: only left child

        successor = find_min(node.right) # case 3: two children
        node.value = successor.value # copy successor's value
        node.right = delete(node.right, successor.value)

    return node

def find_min(node):
    while node.left is not None: # walk left until you can't
        node = node.left
    return node
```

"Same walk as insert. Three cases at the match. One recursive cleanup for the hard case."

▪ SHAPE DECIDES SPEED ▪

Search cost is **O(h)**, where h is the height of the tree. So the question is: how tall is the tree?

A balanced tree is **O(log n)** tall. A degenerate tree is n tall. Same values, different shapes, very different speeds.



[01] Balanced BST
Height ~$\log_2(n)$.
Search cost **O(log n)** ✨

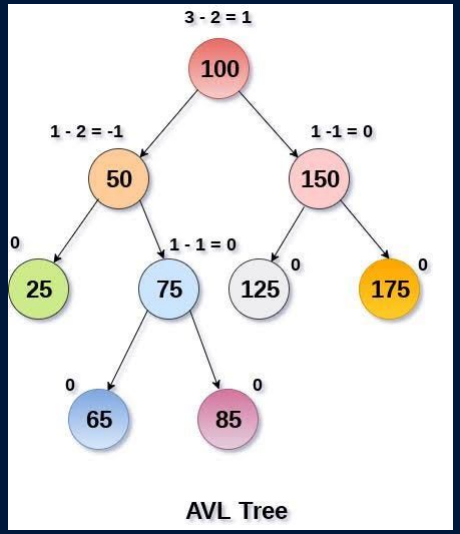
[02] Degenerate (chain)
Height n. Search cost **O(n)** 😢

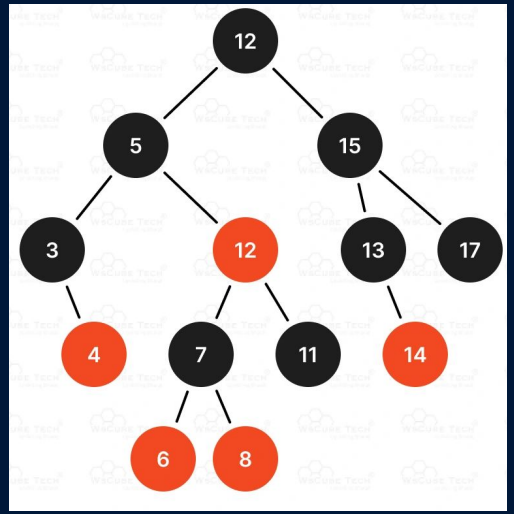
"O(h) — h is the height. Keep the tree short and the BST flies. Let it stretch and it crawls."

SELF-BALANCING TREES (CONCEPT ONLY)

To avoid degenerate trees, smart people invented self-balancing BSTs: trees that automatically rebalance themselves after each insert and delete.

You won't implement them this semester, just know they exist, where they live, and that engineers at Alibaba, ByteDance, and DeepSeek rely on them every day through the standard library.





[01] AVL trees
Strict height balancing — every subtree's left and right heights differ by at most one.

[02] Red-Black trees
Restricts imbalance via strict coloring rules (e.g., no consecutive red nodes, equal black nodes from root to leaves)

[03] B-trees
Keeps all leaves perfectly aligned by growing upward; overfilled nodes split and push their middle key to the parent.

"Plain BST = the principle. Self-balancing BST = the production version. You will use the second."